

4.1 `=` 赋值运算符（递归展开）

- 右侧不立刻求值；只在使用变量时再展开。
- 适合依赖其它变量且想随其变化的场景。

```
make

A = $(B) world
B = hello
msg: ; @echo $(A)      # 输出: hello world
B = hi
again: ; @echo $(A)    # 输出: hi world (跟随 B 的最新值)
```

 复制代码

4.2 `:=` 赋值运算符（立即展开）

- 赋值时就把右侧计算好并存储为常量。
- 适合包含函数结果、通配收集结果等不会再变的值，效率更高。

```
make

SRC := $(wildcard src/*.c)
CFLAGS := -O2 -Wall
```

 复制代码

4.3 `?=` 赋值运算符（若未设置则赋值）

- 只在变量当前未定义时生效，常用来给可覆盖的变量提供默认值。

```
make

CC ?= gcc      # 命令行或环境里没给 CC 时才用 gcc
CFLAGS ?= -O2
```

 复制代码

4.4 `+=` 追加赋值

- 在已有值后面追加内容；对 `=` 和 `:=` 的变量均可用。
- 对递归变量：右侧保持“惰性”；对立即变量：右侧立即求值。

```
make

CFLAGS := -O2
CFLAGS += -Wall      # 现在 CFLAGS 是 -O2 -Wall

LIBS = -lm
LIBS += $(EXTRA)      # 使用时才看 EXTRA 的值
```

 复制代码

5 Makefile 中的自动变量

自动变量只能在**配方（recipe）**里用，随当前规则的目标/依赖而变。

- `$@`：目标文件名（如果是归档成员，形如 `lib.a(member)`）
- `$<`：第一（个）先决条件（首个依赖）。多见于单输入规则（如 `.c -> .o`）
- `$^`：所有先决条件，**去重**，以空格分隔
- `$+`：所有先决条件，**不去重**，顺序保留（链接时可能需要）
- `$?`：比目标新的先决条件（常用于增量构建）
- `$*`：模式规则里的“茎”（stem），即 `%` 匹配到的部分
- 目录/文件片段：`$(@D)`、`$(@F)`、`$(*D)`、`$(*F)`、`$(<D)`、`$(<F)`
- 归档成员：`$(%)`（当目标是 `lib.a(m.o)` 时的成员名）
- 顺序仅依赖：`$(|)`（order-only prerequisites）

6.1 abspath

把路径转为**绝对路径**，不做存在性检查，不解析符号链接。

```
make

ROOT := $(abspath ..)      # ../ -> /full/path/..
```

 复制代码

6.2 addprefix

给一组单词**加前缀**。

```
make

SRC := a.c b.c
SRC_PATH := $(addprefix src/, $(SRC)) # -> src/a.c src/b.c
```

 复制代码

6.3 addsuffix

给一组单词**加后缀**。

```
make

NAMES := foo bar
OBSJS := $(addsuffix .o, $(NAMES)) # -> foo.o bar.o
```

 复制代码

6.4 basename

去掉文件名的**扩展名**（仅最后一个后缀）。

```
make

FILES := a.c b.cc c.tar.gz
BASE := $(basename $(FILES)) # -> a b c.tar
```

 复制代码

6.5 dir

取目录部分（保留末尾斜杠；纯文件名则得 `./`）。

```
make

$(dir src/a.c)      # -> src/
```

[复制代码](#)

6.6 notdir

取非目录部分（文件名）。

```
make

$(notdir src/a.c)    # -> a.c
```

[复制代码](#)

6.7 shell

执行外部命令并把标准输出作为结果（注意可移植性与性能）。

```
make

GIT_HASH := $(shell git rev-parse --short HEAD)
```

[复制代码](#)

小心： `shell` 每次都会启动子进程；若结果固定，配合 `:=` 缓存。

6.8 wildcard

用通配符收集文件列表；比直接在变量里写 `*.c` 更稳妥。

```
make

SRC := $(wildcard src/*.c)      # 空结果也安全
OBJ := $(patsubst %.c,%.o,$(SRC)) # 常和 patsubst 连用（外延函数）
```

[复制代码](#)

相关高频函数（顺手记）：`patsubst`、`filter`/`filter-out`、`foreach`、`sort`、`wordlist`、`subst`、`call`。

7 Makefile 的条件分支

条件求值在 **解析阶段** 进行。

```
make

ifeq ($(CC),clang)
    CFLAGS += -Weverything
else ifneq ($(findstring gcc,$(CC)),)
    CFLAGS += -Wextra
else
    $(warning Unknown compiler $(CC))
endif
```

 复制代码

常用形式：

- `ifeq (A,B)` / `ifneq (A,B)`：字符串比较。**注意空格**，常把被比对项放在函数结果里避免多余空格。
- `ifdef VAR` / `ifndef VAR`：是否已定义（非空）。
- `else` / `endif`：配套结束。

技巧：

- 写法更稳的比较：`ifeq ($(strip $(MODE)),release)`（去掉前后空白）
- 条件里可调用函数；但不要在条件里做昂贵的 `shell`。

8 Makefile 的互相包含

把其它 Makefile 拉进来，便于拆分配置。

```
make

# 基本用法
include config.mk rules.mk

# 忽略缺失：用 -include 或 sinclude
-include local.mk
```

 复制代码

要点：

- 被包含文件的**相对路径**相对当前 Makefile 所在目录（不是工作目录）。
- 可用 `wildcard`/变量拼出名单：`include $(wildcard conf/*.mk)`
- 如果希望缺失不报错：`-include` 或 `sinclude`
- 变量、规则在包含后汇入同一命名空间；重复定义后者覆盖前者（遵循解析顺序）。

9 Makefile 的运行

基本形式：`make [选项] [目标] [变量=值 ...]`

常用选项速记：

- `-f FILE`：指定 Makefile（默认 `Makefile / makefile`）
- `-C DIR`：先切换目录再执行
- `-j [N]`：并行度（不带 N 表示不限制；注意配方里的临界资源）
- `-n`：只打印将要执行的命令，不执行（dry-run）
- `-B`：强制重建所有目标（忽略时间戳）
- `-k`：出错后继续其它独立目标
- `-s`：静默模式（不回显命令）
- `--warn-undefined-variables`：引用了未定义变量时给警告（调试利器）

命令回显与错误：

- 默认会回显命令；在单行前加 `@` 抑制：`@echo building...`
- 在命令前加 `-` 忽略该行的非零返回码：`-rm -f *.o`

扩展函数使用方式

patsubst

签名：`$(patsubst pattern,replacement,text)`

作用：把 `text` 中按词（空白分词）匹配 `pattern` 的词，替换成 `replacement`。`%` 是通配符（任意“茎”）。

例子

```
make 📄 复制代码

SRC := foo.c bar.c lib/baz.cc
OBJ := $(patsubst %.c,%.o,$(filter %.c,$(SRC))) # -> foo.o bar.o
CPP := $(patsubst %.cc,%.o,$(filter %.cc,$(SRC))) # -> lib/baz.o
# 路径映射
BUILD_OBJ := $(patsubst src/%.c,build/%.o,$(wildcard src/*.c))
```

要点/坑

- 只对整词生效；不匹配的词原样保留。
- `pattern/replacement` 可含目录：`src/%.c -> build/%.o`。
- 不是正则，仅 `%` 通配。
- 不去重；如需去重再用 `$(sort ...)`。

filter / filter-out

签名:

- `$(filter pattern...,text)`: 保留匹配这些 `pattern` 的词
 - `$(filter-out pattern...,text)`: 剔除匹配的词
- 同样使用 `%` 通配, 逐词匹配。

例子

```
make 📄 复制代码

FILES := a.c b.cc c.c d.hpp
C      := $(filter %.c,$(FILES))           # -> a.c c.c
CXXH   := $(filter %.hpp %.hh,$(FILES))    # -> d.hpp
NOTC   := $(filter-out %.c,$(FILES))        # -> b.cc d.hpp
# 组合: 选出第三方库以 lib 开头的 .a
THIRDPARTY := $(filter lib%/%.a,$(LIBS))
```

要点/坑

- 匹配是大小写敏感的单词匹配。
- 多个 `pattern` 可并列: `%.c %.cc`。
- 想做“集合差/交/并”, 用 `filter / filter-out + sort` 很顺手。

foreach

签名: `$(foreach var,list,text)`

作用: 把 `var` 依次设为 `list` 的每个词, 展开 `text`, 把所有结果用空格串起来返回。`var` 在循环内像“局部变量”。

例子

```
make 📄 复制代码

# 为每个目录生成一个 -I 开关
INCLUDE_DIRS := include thirdparty/foo/include
CPPFLAGS += $(foreach d,$(INCLUDE_DIRS),-I$(d))

# 生成一组目标: src/x.c -> build/x.o
SRC := $(wildcard src/*.c)
OBJ := $(foreach s,$(SRC),build/$(notdir ${s%.c}.o))
```

要点/坑

- 循环结果之间默认用空格相连; 想换行可手动插入换行符 (定义变量 `nl := \n` 再 `$(nl)` 使用)。
- `var` 的旧值在循环后会恢复。
- 复杂生成 (规则/变量) 经常配合 `eval` 使用 (进阶)。

sort

签名: `$(sort list)`

作用: 按字典序排序并去重 (还会把多余空白压成单空格)。

例子

```
make

PATHS := b a a c
UNIQ  := $(sort $(PATHS))      # -> a b c
# 做“集合并”时常用
UNION := $(sort $(A) $(B))
```

 复制代码

要点/坑

- 仅按字节序 (ASCII) 排序; 不是自然数序。
- 会去重; 若只想排序不去重, GNU Make 没有原生函数 (一般自己实现意义不大)。

wordlist

签名: `$(wordlist s,e,text)` (1 基索引, 包含端点)

作用: 从 `text` 中取第 `s` 到第 `e` 个词。

例子

```
make

LIST := a b c d e
MID  := $(wordlist 2,4,$(LIST))      # -> b c d
HEAD3 := $(wordlist 1,3,$(LIST))     # -> a b c
TAIL2 := $(wordlist 4,100,$(LIST))   # -> d e
```

 复制代码

要点/坑

- 索引越界不报错; `e` 超长会截到最后, `e < s` 得空。
- 相关三兄弟: `words $(LIST)` (词数), `word n,$(LIST)` (第 `n` 个词)。

subst

签名: `$(subst from,to,text)`

作用: 在 `text` 里把所有 `from` 子串替换为 `to` (非按词, 也不支持 `%` 模式)。

例子

```
make

X := a-b-c
Y := $(subst -,_,$(X))           # -> a_b_c
# 移除所有空格
NOSPACE := $(subst $(space),,$(SOME_TEXT))
```

[复制代码](#)

要点/坑

- 是纯文本全局替换, 不是模式匹配。想按扩展名映射, 用 `patsubst`。
- 参数用逗号分隔; 若需要逗号, 可先定义变量:

```
make

comma := ,
STR := $(subst $(comma),,,$(CSV))
```

[复制代码](#)

- 与 `strip` 搭配常用: `$(strip $(subst \n, ,$(TEXT)))`。

call

签名: `$(call func,arg1,arg2,...)`

作用: 把变量 `func` 里的“模板”当作自定义函数来调用, 模板中可使用位置参数 `$(1) $(2) ...`。

关键点: `call` 会先展开参数, 再把参数值填入模板并展开模板。

例子 1: 定义一个“映射”函数

```
make

# 取 .C/.cc -> .o
map-to-o = $(patsubst %.c,%.o,$(filter %.c,$(1))) \
           $(patsubst %.cc,%.o,$(filter %.cc,$(1)))

OBJ := $(call map-to-o,$(SRC_FILES))
```

 复制代码

例子 2: 生成规则 (配合 eval)

```
make

# 模板: 给目录生成安装规则
define make-install-rule
.PHONY: install-$(1)
install-$(1):
    @install -d $(DESTDIR)/$(1)
    @cp -r $(2) $(DESTDIR)/$(1)/
endef

$(eval $(call make-install-rule,bin,$(BINARIES)))
$(eval $(call make-install-rule,share,$(DATADIR)))
```

 复制代码

要点/坑

- 模板里的逗号、括号要注意与 `call` 的实参分隔不冲突 (必要时把大段模板放到 `define ... endef`)。
- `call` 本身不改变延迟/立即展开语义; 若结果固定, 最好用 `:=` 缓存:
`RESULT := $(call heavy,$(ARGS))`。

```
$(foreach var, list, text)
```

核心语义（4 步）

1. **分词**：把 `list` 按空白（空格/Tab/换行）切成词。
2. **赋值**：依次把循环变量 `var` 设为每个词。
3. **展开**：用当前 `var` 展开 `text`。
4. **拼接**：把每次展开结果用单个空格连接并返回。

变量与作用域

- `var` 名字任意（如 `x/d/file`）；在循环内临时生效。
- 循环结束后，外层同名变量会恢复原值（遮蔽后还原）。

分隔符技巧

- 默认用空格拼接；想用别的分隔：

```
make

comma := ,
CSV := $(foreach x,$(LIST),$(x)$(comma))    # a, b, c,
define NL

endif
LINES := $(foreach x,$(LIST),$(x)$(NL))      # a\nb\nc\n
```

[复制代码](#)